



IMT Atlantique

Bretagne-Pays de la Loire
École Mines-Télécom



PROCOM: AutoMashup - Mashups musicaux, améliorés par les outils d'IA

Soutenance finale

Projet 39:

- Assié Quentin
- Lbakali Fouad
- Marchal Loick
- Souissi Mouad

Encadré par:

- Axel Marmoret
- Bastien Pasdeloup

Présentation du projet

Présentation du projet

- Automashup est un algorithme générant automatiquement des Mashups, en associant la partie vocale d'une chanson "A" à la partie instrumentale d'une chanson "B", et dont les bases de développement ont été posées précédemment lors de 2 ProCom successifs.

The AutoMashup Pipeline

AutoMashup aligns two songs harmonically and rhythmically to create a coherent mashup. The process involves source separation, music analysis, and alignment.

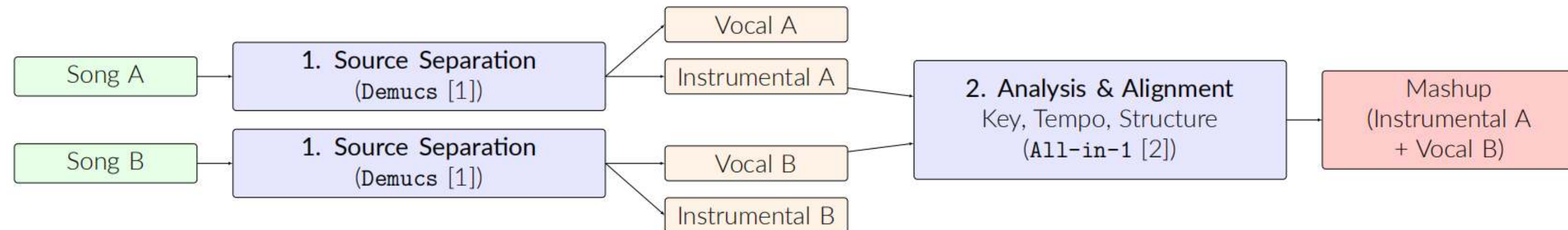


Figure 1. The AutoMashup workflow, from input songs to final mashup.

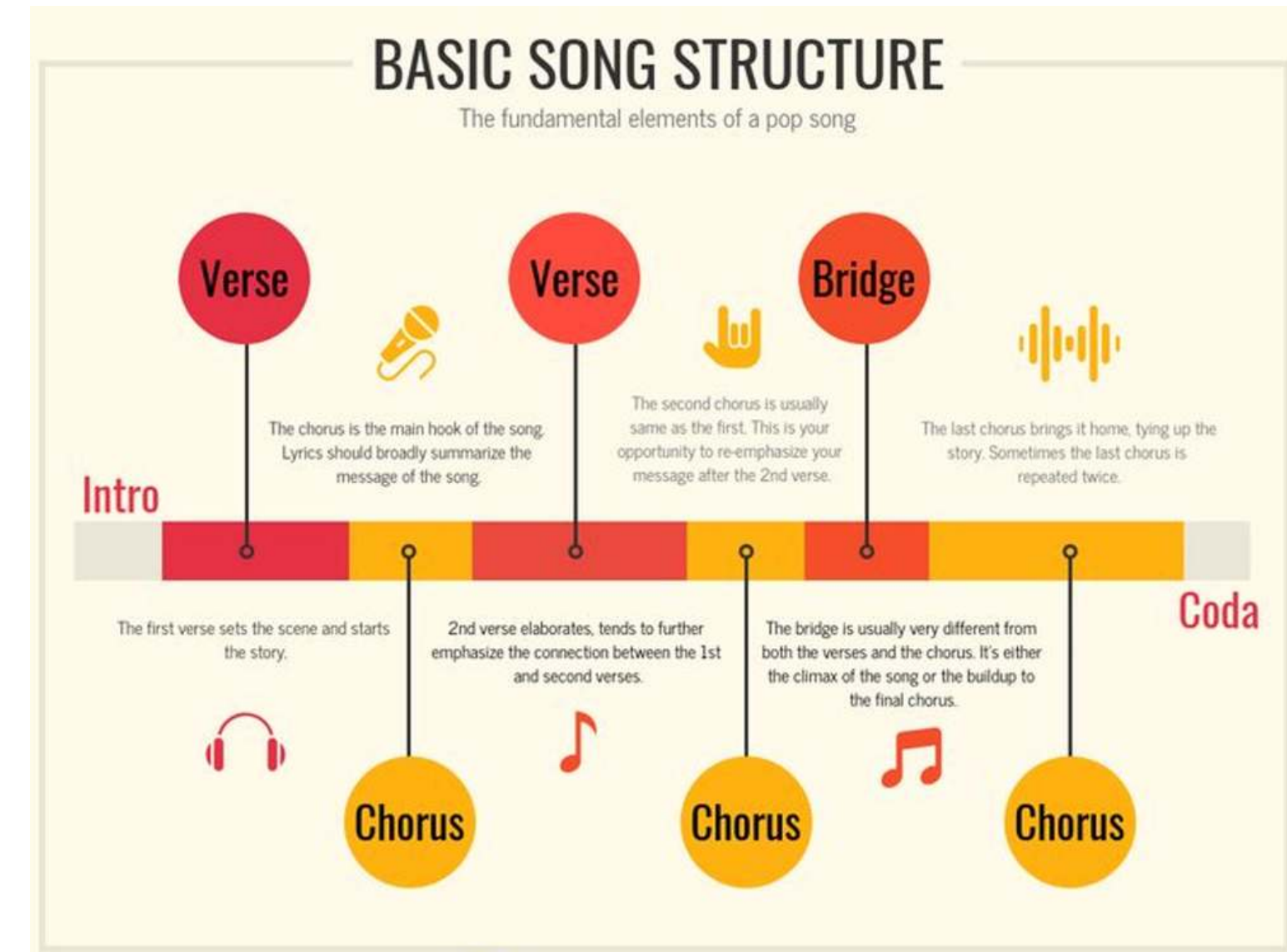
Présentation du projet

- Le projet s'inscrit dans le cadre des activités de recherche de l'équipe **Brain** du département **MEE** à l'IMT Atlantique, spécialisé dans le **Machine Learning** et le **Deep Learning** appliqués aux signaux temporels.
- Dans ce contexte, le projet vise le **traitement automatisé de la musique**, considérée comme un signal acoustique complexe, et se rattache au domaine en plein essor de l'**Informatique Musicale (Music Information Retrieval)**

Objectifs du projet

Résoudre deux limitations majeures de la version actuelle d'AutoMashup:

- Pb1.Silences et désalignements structurels: Les chansons générées sont alignées en fonction de leur structure. Cela implique qu'il y a parfois des silences, lorsque la structure n'est pas la même entre les 2 morceaux. Ces silences apparaissent soit dans une piste instrumentale, soit dans la voix, soit dans les deux.
- Pb2.Détérioration de la qualité audio: La séparation de sources, l'alignement en tempo et en tonalité détériorent grandement la qualité musicale des mashups, par rapport aux chansons initiales



Amélioration de la qualité audio

ARCHITECTURE DU SYSTÈME:

- SÉPARATION : Demucs / allin1
- MODÉLISATION : Piste / Segment
- ALIGNEMENT : Synchronisation du rythme et du tempo
- INPAINTING : Gestionnaire de silence
- AMÉLIORATION : Débruitage
- TRANSITIONS : Lissage
- MASTERING : LUFS / Limiteur
- CONTRÔLE QUALITÉ : Score et métriques

PHASE 1: Préparation & Alignement

1. PRÉ-TRAITEMENT

- SourceSeparator divise les chansons en stems (pistes séparées) à l'aide de Demucs.
- QualityAssessment évalue immédiatement le RSB/RMS. Les stems à faible indice de confiance sont signalés pour une nouvelle séparation.

2. MODÉLISATION DES PISTES

- L'audio, les rythmes et les métadonnées sont chargés dans des objets Track.
- Le détecteur de silence intègre des marqueurs, fournissant aux étapes suivantes un contexte crucial sur les silences.

3. ALIGNEMENT

- MashupEngine sélectionne une référence.
- BeatSynchronizer et TempoMatcher verrouillent la phase et le BPM.
- Le correcteur de tonalité (Pitch Corrector) unifie la tonalité.

4. SILENCE ET INPAINTING :

- SilenceHandler classe les vides.
- Les vides courts utilisent des fondus enchaînés (crossfades).
- Les vides moyens utilisent WaveNet ; les longs vides utilisent le modèle génératif AudioLDM.

5. AMÉLIORATION PAR STEM

- Audio Enhancer élimine le bruit.
- Vocal Enhancer clarifie la voix (dé-esseur).
- Instrumental Enhancer restaure les transitoires et élargit la stéréo.

PHASE 3: Production et Finalisation

6. MIXAGE DES STEMS :

- StemMixer achemine les voix et les instruments via des chaînes dédiées.
- AutoMixer applique une égalisation (EQ), une compression et un traitement spatial spécifiques à chaque piste avant la sommation.

7. TRANSITIONS

- TransitionSmoother gère les limites entre les segments.
- Il optimise les fondus enchaînés et applique un lissage spectral pour éviter les sauts de timbre abrupts entre les clips

8. **FINALISATION:** Le Mastering garantit les standards de volume sonore (loudness) commerciaux, normalise le volume via pyloudnorm, et applique l'égalisation et la limitation finales.

Bilan

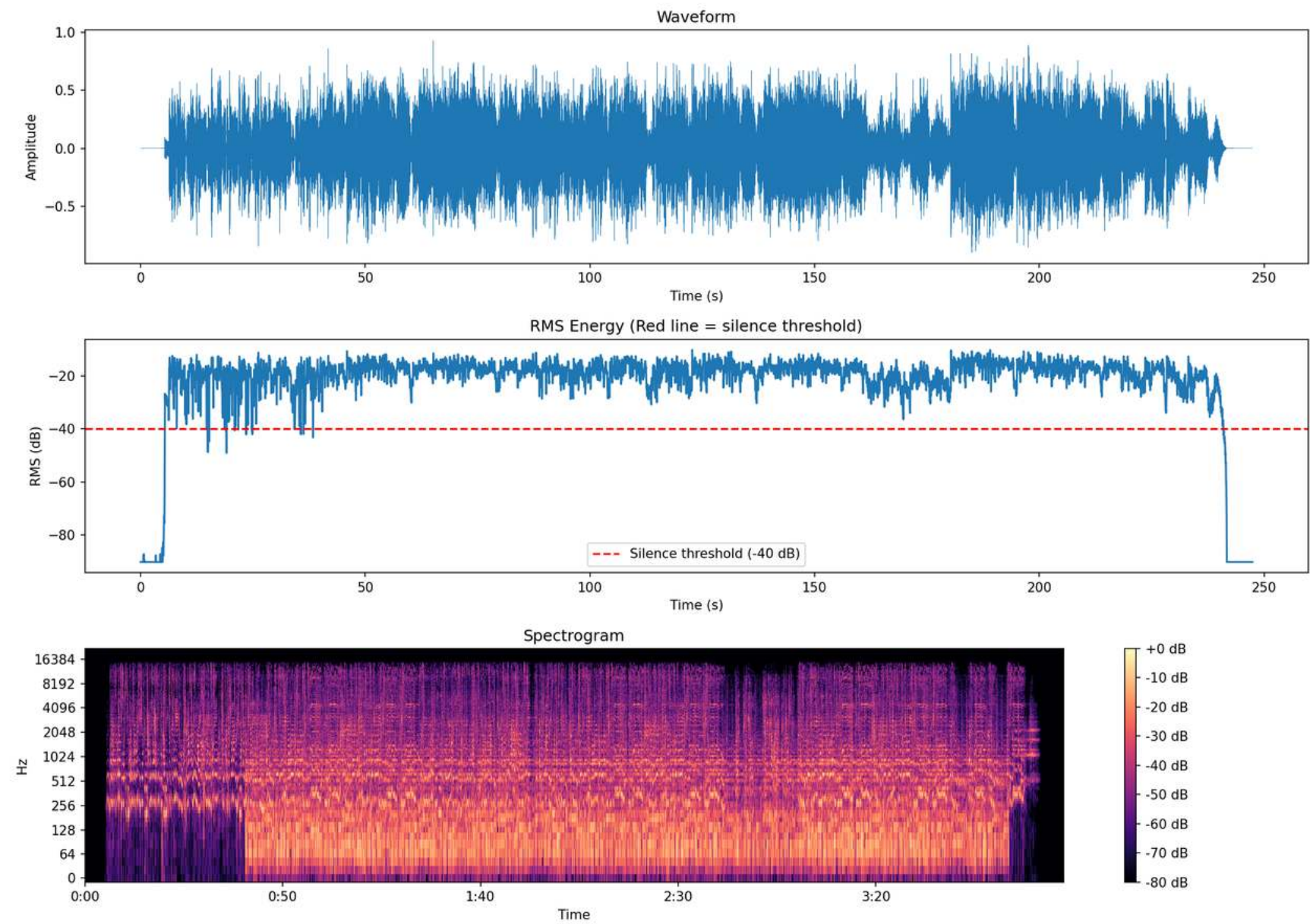
- + Prise en main d'AutoMashup
- + Amélioration de la qualité (notamment de la partie vocale)
- Problèmes d'alignement (non résolus)
- Approche abandonnée



Autres features

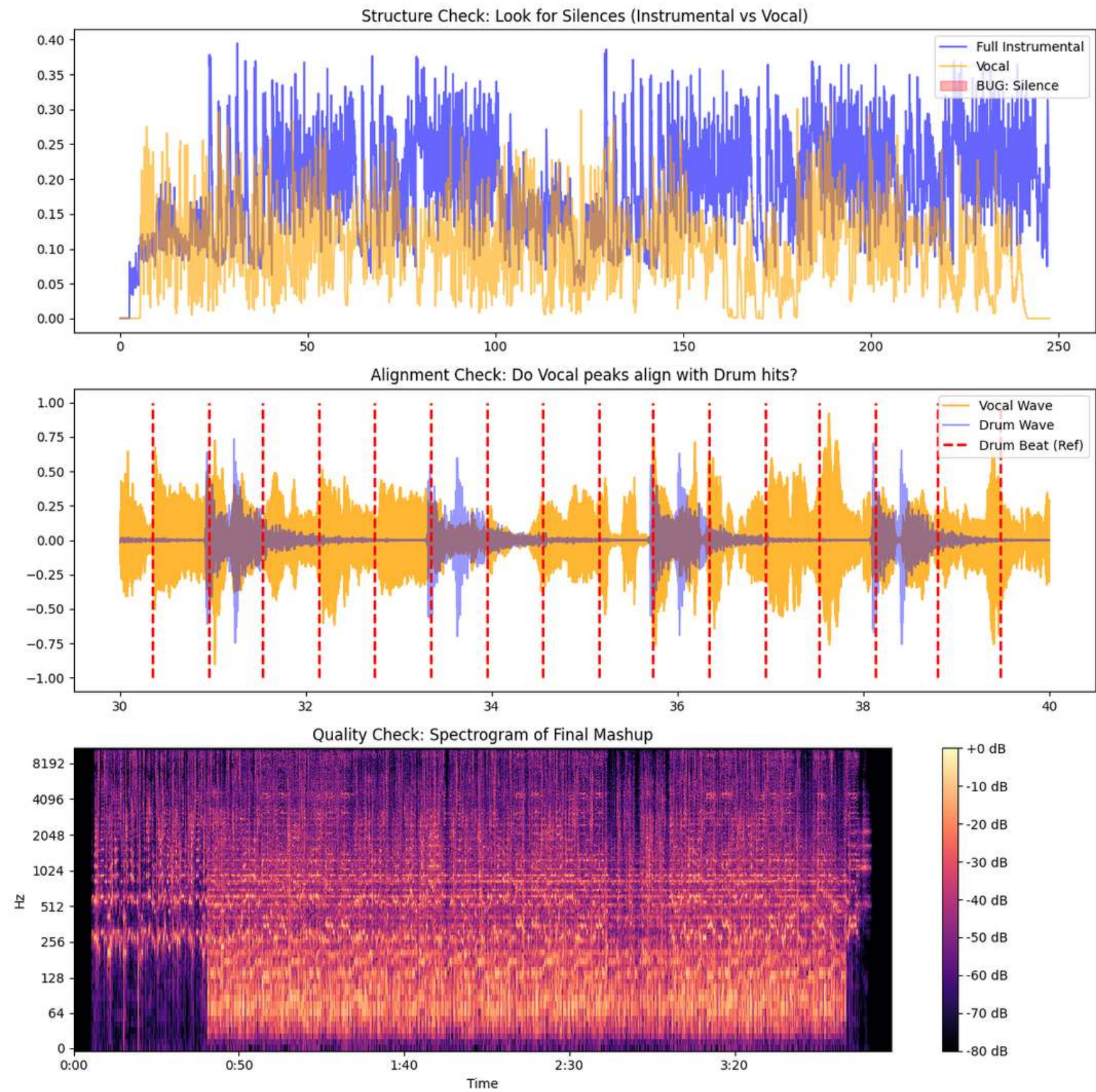
Features implémentées

- Détecteur de silence
- Spectrogramme



Features implémentées

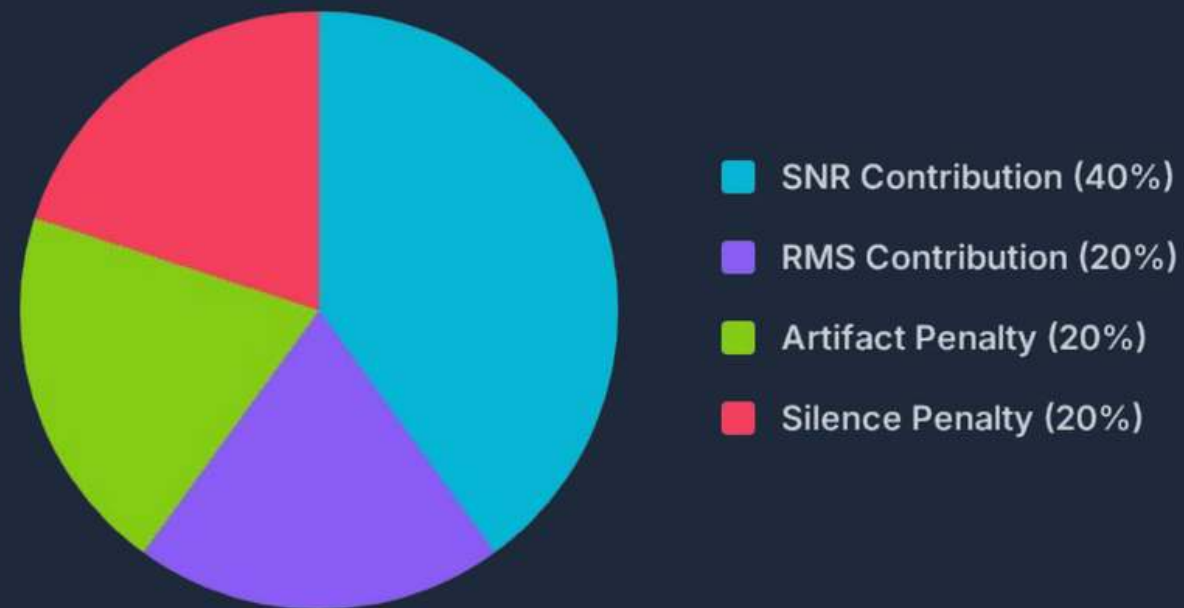
- Structure & Alignement
- Spectrogramme



Features implémentées

Score de qualité et métriques utilisées

QUALITY SCORE LOGIC



Pass Threshold: ≥ 0.8 is "Release Ready". < 0.7 triggers automated reprocessing.

SNR (SIGNAL-TO-NOISE RATIO)

Measures separation clarity. Values $> 30\text{dB}$ are good.

$$10 \times \log_{10} \frac{\text{mean}(\text{audio}^2)}{\text{noise}_{\text{floor}}}$$

RMS (ROOT MEAN SQUARE)

Approximates perceived loudness and headroom.

$$\sqrt{\text{mean}(\text{audio}^2)}$$

Target: 0.1 - 0.2 (Healthy Headroom)



ACE-STEP

Modèle

- ACE-Step, un nouveau foundation model open-source pour la music generation qui surpasse les limitations clés des approches existantes et atteint des performances state-of-the-art grâce à un design architectural holistique.
- Les anciennes méthodes font face à des compromis inhérents entre generation speed, musical coherence et controllability.
- ACE-Step comble cet écart en intégrant la diffusion-based generation avec le Deep Compression AutoEncoder (DCAE) de Sana et un linear transformer léger. Il exploite également MERT et m-hubert pour aligner les semantic representations (REPA) pendant le training, permettant une convergence rapide.
- Plutôt que de construire un autre pipeline text-to-music end-to-end, la vision des créateurs d'ACE-Step est d'établir un foundation model pour l'IA musicale : une architecture rapide, polyvalente, efficace et flexible qui facilite l'entraînement de sub-tasks par-dessus.



En pratique

- **Paroles:** Les paroles sont choisies au hasard au sein de la communauté de AI music generation ou sur internet et ne font pas partie du training set d'ACE-STEP
- **Instrumentaux :**
 - Supporte la génération de diverses musiques instrumentales à travers différents genres et styles.
 - Capable de produire des pistes instrumentales réalistes avec le timbre et l'expression appropriés pour chaque instrument.
 - Peut générer des arrangements complexes avec plusieurs instruments tout en maintenant la musical coherence.
- **Controllability:**
 - Le modèle supporte diverses applications training-free.
 - retake : régénère une variation de la même chanson.
 - repaint : régénère une partie spécifique de la chanson.
 - edit : modifie les paroles de la chanson.



Limitations

- **Output Inconsistency** : Très sensible aux random seeds et à la durée de l'input, menant à des résultats aléatoires.
- **Faiblesses spécifiques à certains styles** : Sous-performe sur certains genres. Adhésion au style limitée et plafond de musicalité.
- **Continuity Artifacts** : Transitions non naturelles dans les opérations de repainting / extend.
- **Vocal Quality** : Parfois on assiste à une synthèse vocale grossière manquant de nuances.
- **Control Granularity** : Nécessite un contrôle plus fin des paramètres musicaux.



Amélioration qualité par mixage

1) Méthodes sans algorithmes d'apprentissage

Objectif : Améliorer l'intégration du vocal séparé dans l'instrumental après repitch / time-stretch, sans recourir à un modèle d'apprentissage.

Problèmes observés :

- Artefacts de séparation de source
- Dureté dans les aigus / effet "granuleux"
- Mauvaise fusion voix-instrumental

Approche retenue

- traitement audio classique (DSP)
- sans réseau de neurones
- chaîne d'effets légère appliquée au vocal :
 - filtres , low pass , h p , ect
 - distortion légère
 - reverb courte , delay



Passage en stéréo du pipeline

- Remplacement du chargement mono pour conserver les 2 canaux
- Adaptation des fonctions de durée/alignment (pad, crop, mix) pour gérer les formats mono (n,) et stéréo (n,2)
- Objectif : préserver l'image stéréo jusqu'au mix final

Intérêt

- simple à intégrer dans le pipeline
- peu coûteux en calcul
- facilement réglable à l'écoute
- permet de masquer une partie des artefacts sans réentraîner de modèle

➡ Avant d'utiliser des méthodes d'auto-mix avancées, une chaîne de mixage DSP permet déjà d'améliorer la qualité perçue.

1) Méthodes avec algorithmes d'apprentissage

Objectif : Améliorer l'intégration du vocal séparé dans l'instrumental après repitch / time-stretch, cette fois en utilisant des modèles de deep learning disponibles.

DFN / DeepFilterNet

- utilisé sur la basse dans certains cas de instrumental → vocal repitch
- objectif : réduire une saturation/perception trop agressive après transposition
- effet observé : basse plus lissée, un peu plus propre et mieux intégrée au mix

MSG

- testé surtout sur la voix dans le cas vocals → instrumental
- effet observé : voix un peu plus spatiale, large et légèrement plus agréable
- limite : gain qualitatif modéré dans mon pipeline

Conclusion

- les méthodes apprenantes peuvent apporter une amélioration ponctuelle
- mais dans mon cas, le gain reste souvent comparable ou inférieur à une bonne chaîne DSP simple
- intérêt principal : raffinement perceptif, pas transformation majeure du rendu comme attendu :(

2) Pistes d'amélioration

Approches sans apprentissage

- Etendre la chaîne d'effets DSP au cas adjust by section voix → instrumentale
- Combiner un meilleur alignement structurel avec un masquage local des artefacts
- Appliquer la chaîne DSP sur repitch instru to vocal (pas encore testé)

Approches avec apprentissage

- MSG sur l'instrumentale dans les cas de repitch de l'instru, et non seulement sur la voix
- combiner MSG + chaîne DSP sur voix et/ou instru pour mieux couvrir les artefacts résiduels
- explorer AutoMixToolkit pour des stratégies de mixage automatique au niveau des stems
- explorer Matchering pour améliorer le rendu final par matching de référence sur le mix stéréo

Objectif global : mieux articuler l'alignement musical , la correction d'artefacts et la finition du rendu final

3) Bonus

Objectif : faciliter l'utilisation du pipeline sans passer par des scripts manuels

AutoMashup - imta

Choisis 2 fichiers WAV, choisis la technique de mashup, puis lance.

VOCAL (WAV)

INSTRUMENTALE (WAV)


Déposez le fichier ici
- ou -
Cliquez pour télécharger un fichier


Déposez le fichier ici
- ou -
Cliquez pour télécharger un fichier

BPM morceau 1 (optionnel)

BPM morceau 2 (optionnel)

0

0

Technique de mashup

Section alignment downbeats — instrumentale vers voix

Dossier de sortie

/home/quentin/Documents/song/mashups_created

Lancer le mashup

Statut

Fonctionnalités

- import de deux fichiers WAV : voix et instrumentale
- choix de la technique de mashup
- possibilité d'entrer manuellement le BPM des morceaux
- lancement automatique du traitement
- écoute et récupération directe du fichier généré

Intérêt

- interface plus simple pour tester rapidement plusieurs configurations
- gain de temps pour les comparaisons

APPROCHES ABANDONNÉES

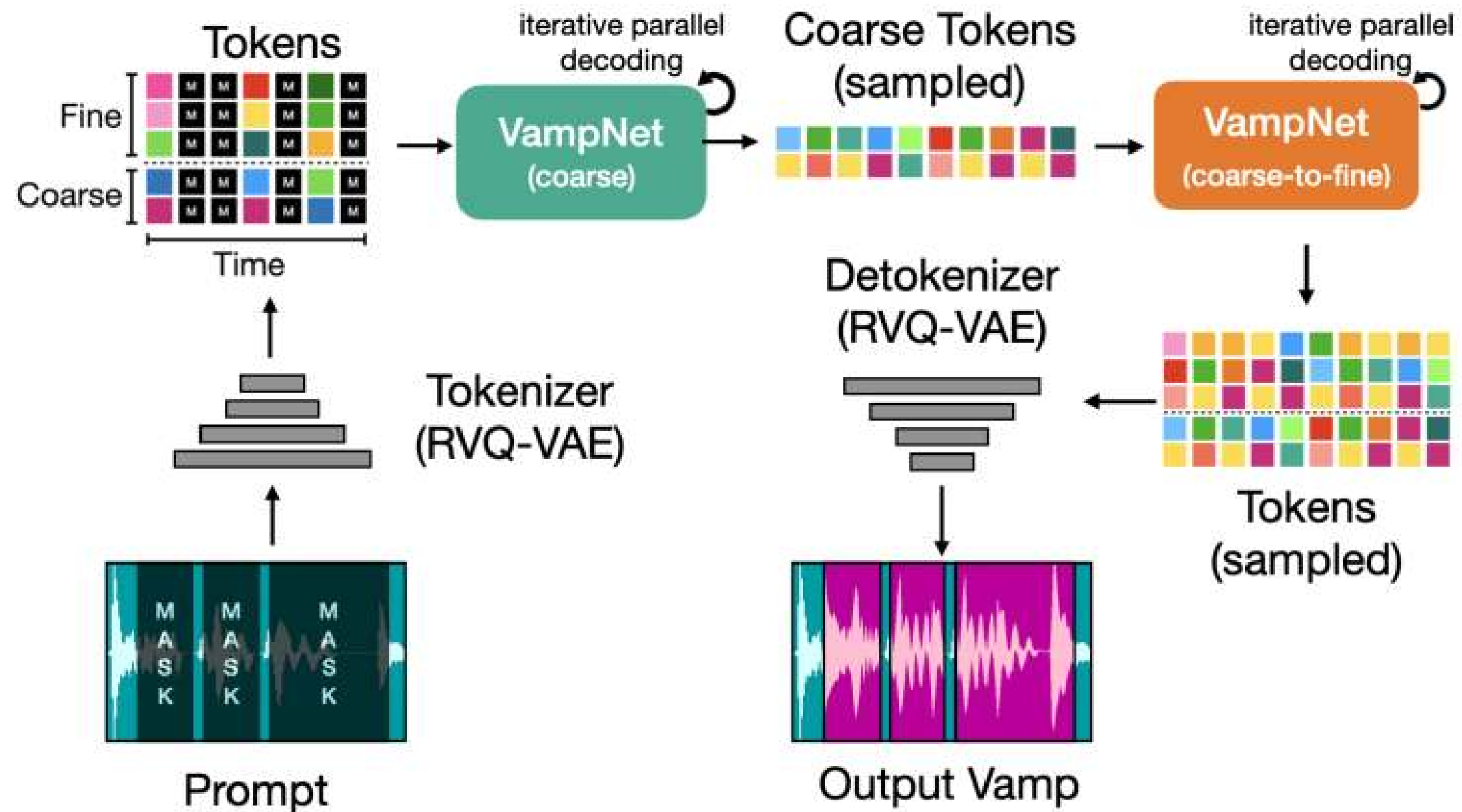
Compatibility Predictor

- **L'Objectif:**

- Développer un modèle qui reçoit une Vocal loop et une Instrumental loop et génère un score (de 0 à 1) indiquant leur compatibilité musicale. Ce processus utilise des embeddings plutôt qu'un traitement lourd de l'raw audio.

- **Quality** : Le système rejette les mauvais appariements avant même d'appliquer trop de stretch ou de pitch. Cela s'apparente au rôle de "gatekeeper" du module Quality Assurance qui déclenche un retraitement si le score est insuffisant.
- **Alignment** : Le modèle peut être entraîné pour prédire le meilleur offset (shift) afin de verrouiller les voix sur le beat. Cela permet de résoudre le problème d'alignment en amont des étapes de Beat & Tempo Sync

VampNet

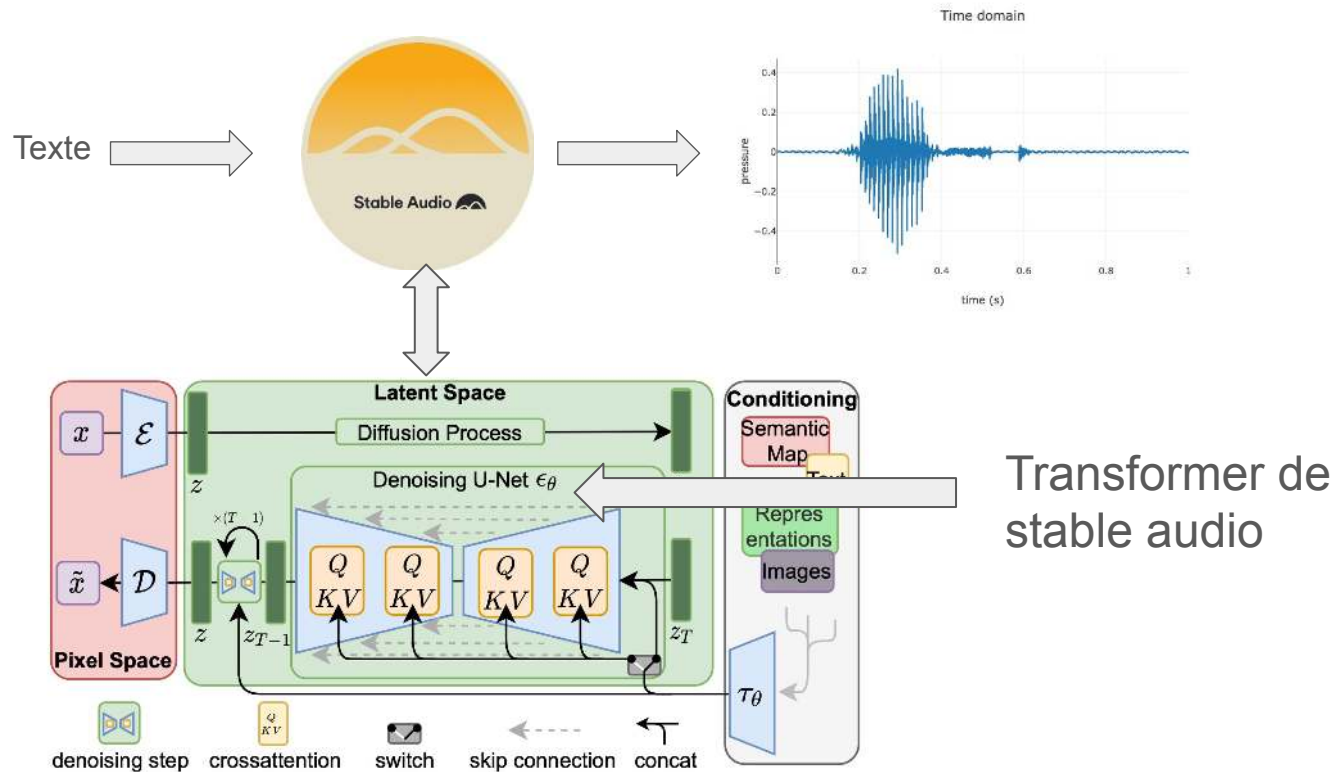


<https://arxiv.org/pdf/2307.04686>

MERCI

Le Modèle Stable Audio

Stable audio



SMLD

$$\mathbf{x}_\tau = \mathbf{x}_0 + \sigma(\tau)\boldsymbol{\epsilon}, \text{ where } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

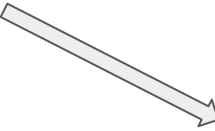
$$\sigma(0) = 0 \quad \sigma(T) = \sigma_{max} \quad x_0 \sim p_{data} \quad x_T \sim \mathcal{N}(0, \sigma_{max}I_d)$$

Forward SDE

ODE

$$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}} d\mathbf{w} \quad d\mathbf{x} = -\dot{\sigma}(\tau)\sigma(\tau)\nabla_x \log p_\tau(\mathbf{x}_\tau) d\tau$$

Le transformer de stable audio prédit


$$\nabla_x \log p_\tau(x) = \frac{\mathbb{E}[x_0 | x] - x}{\sigma^2}$$

Le transformer

```
class StableAudioDiTModel(ModelMixin, AttentionMixin, ConfigMixin):
```

```
    num_layers: int = 24
```

```
    for block in self.transformer_blocks:
        hidden_states = block(
            hidden_states=hidden_states,
            attention_mask=attention_mask,
            encoder_hidden_states=cross_attention_hidden_states,
            audio_hidden_states=cross_attention_audio_states,
            encoder_attention_mask=encoder_attention_mask,
            rotary_embedding=rotary_embedding,
        )
```

Le transformer block

```
class StableAudioDiTBlock(nn.Module)
```

```
    # 0. Self-Attention
```

```
    norm_hidden_states = self.norm1(hidden_states)
```

```
    attn_output = self.attn1(  
        norm_hidden_states,  
        attention_mask=attention_mask,  
        rotary_emb=rotary_embedding,  
    )
```

```
    hidden_states = attn_output + hidden_states
```

```
    # 2. Cross-Attention
```

```
    norm_hidden_states = self.norm2(hidden_states)
```

```
    attn_output = self.attn2(  
        norm_hidden_states,  
        encoder_hidden_states=encoder_hidden_states,  
        attention_mask=encoder_attention_mask,  
    )
```

```
    hidden_states = attn_output + hidden_states
```

```
    # 3. Feed-forward
```

```
    norm_hidden_states = self.norm3(hidden_states)  
    ff_output = self.ff(norm_hidden_states)
```

```
    hidden_states = ff_output + hidden_states
```

```
    return hidden_states
```

Inpainting avec Stable audio

Algorithm 1 Inference Conditioned for Audio Inpainting

Require: observations $\mathbf{y}$, inpainting mask $\mathbf{m}$, number of iterations T , noise schedule τ_i

Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \sigma_{\max}^2 \mathbf{I})$ $\triangleright$ Initial noise is $\mathbf{x}_T$

for $i = T, \dots, 1$ **do** $\triangleright$ Step backwards

$\hat{\mathbf{x}}_0 = D_{\theta}(\tilde{\mathbf{x}}_i, \tilde{\tau}_i)$ $\triangleright$ Denoiser

$\hat{\mathbf{x}}'_0 = \mathbf{y} + (\mathbf{1} - \mathbf{m}) \odot \hat{\mathbf{x}}_0$ $\triangleright$ Data consistency

$\mathbf{x}_{i-1} = \tilde{\mathbf{x}}_i - (\sigma(\tau_{i-1}) - \sigma(\tilde{\tau}_i)) \left(\frac{\hat{\mathbf{x}}'_0 - \tilde{\mathbf{x}}_i}{\sigma(\tilde{\tau}_i)} \right)$ $\triangleright$ Update

step

end for

return $\hat{\mathbf{x}}_0$ $\triangleright$ Output is the reconstructed signal

Inpainting par masking

$$d\mathbf{x} = -\dot{\sigma}(\tau)\sigma(\tau)\nabla_{\mathbf{x}}\log p_{\tau}(\mathbf{x}_{\tau})d\tau$$

$$\nabla_x \log p_{\tau}(x) = \frac{\mathbb{E}[x_0 \mid x] - x}{\sigma^2}$$

Avantage :

- pas coûteux en calcul

Inconvénient :

- pas cohérent mathématiquement

Code

```
mask_start_latent = int((mask_start_in_s * self.vae.config.sampling_rate) / downsample_ratio)
mask_end_latent = int((mask_end_in_s * self.vae.config.sampling_rate) / downsample_ratio)
```

```
noise_pred = self.transformer(
    latent_model_input,
    t.unsqueeze(0),
    encoder_hidden_states=text_audio_duration_embeds,
    global_hidden_states=audio_duration_embeds,
    rotary_embedding=rotary_embedding,
    return_dict=False,
)[0]
```

```
x0_hat = latents - sigma_t*noise_pred
x0_hat = self.proj_convex_set(x0_hat, self.y, self.degradation)
noise_pred = (latents-x0_hat)/sigma_t
```

```
# compute the previous noisy sample  $x_t \rightarrow x_{t-1}$ 
latents = self.scheduler.step(noise_pred, t, latents, **extra_step_kwargs).prev_sample
```

Inpainting score-based

Applying Bayes' rule, the posterior factorizes as $p_{\tau}(\mathbf{x}_{\tau}|\mathbf{y}) \propto p_{\tau}(\mathbf{x}_{\tau})p_{\tau}(\mathbf{y}|\mathbf{x}_{\tau})$, which leads to

$$\nabla_{\mathbf{x}_{\tau}} \log p_{\tau}(\mathbf{x}_{\tau}|\mathbf{y}) = \nabla_{\mathbf{x}_{\tau}} \log p_{\tau}(\mathbf{x}_{\tau}) + \nabla_{\mathbf{x}_{\tau}} \log p_{\tau}(\mathbf{y}|\mathbf{x}_{\tau}),$$

$$\nabla_{\mathbf{x}} \log p_{\tau}(\mathbf{y}|\mathbf{x}_{\tau}) \simeq -\xi(\tau) \nabla_{\mathbf{x}} \|\mathbf{y} - \mathbf{m} \odot \hat{\mathbf{x}}_0\|^2$$

$$\xi(\tau) = \xi' \sqrt{N} / (\sigma(\tau) \|\nabla_{\mathbf{x}} \|\mathbf{y} - \mathbf{m} \odot \hat{\mathbf{x}}_0\|^2\|^2)$$

Calcul de la likelihood

$$y = Ax,$$

et si

$$x \mid x(t) \sim \mathcal{N}(\hat{x}(x(t)), \Sigma),$$

alors

$$y \mid x(t) \sim \mathcal{N}(A\hat{x}(x(t)), A\Sigma A^\top).$$

Calcul de la covariance de $x|x(t)$

Assuming a Gaussian prior $p(x)$ with covariance Σ_x , the covariance $\hat{\Sigma}$ of $p(x | x(t))$ takes the form

$$\hat{\Sigma} = \Sigma_x - \Sigma_x \left(\Sigma_x + \frac{\sigma(t)^2}{\mu(t)^2} I \right)^{-1} \Sigma_x$$

Algorithm 1 Inference Conditioned for Audio Inpainting

Require: observations $\mathbf{y}$, inpainting mask $\mathbf{m}$, number of

iterations T , noise schedule τ_i

Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \sigma_{\max}^2 \mathbf{I})$ $\triangleright$ Initial noise is $\mathbf{x}_T$

for $i = T, \dots, 1$ **do** $\triangleright$ Step backwards

$\hat{\mathbf{x}}_0 = D_{\theta}(\tilde{\mathbf{x}}_i, \tilde{\tau}_i)$ $\triangleright$ Denoiser

$\hat{\mathbf{x}}_0 = \hat{\mathbf{x}}_0 - \sigma(\tilde{\tau}_i)^2 \xi(\tilde{\tau}_i) \nabla_{\tilde{\mathbf{x}}} \|\mathbf{y} - \mathbf{m} \odot \hat{\mathbf{x}}_0\|^2$ $\triangleright$ Eq. (7)

$\mathbf{x}_{i-1} = \tilde{\mathbf{x}}_i - (\sigma(\tau_{i-1}) - \sigma(\tilde{\tau}_i)) \left(\frac{\tilde{\mathbf{x}}_0 - \tilde{\mathbf{x}}_i}{\sigma(\tilde{\tau}_i)} \right)$ $\triangleright$ Update

step

end for

return $\hat{\mathbf{x}}_0$ $\triangleright$ Output is the reconstructed signal

Code

```
if self.do_score_guidance :  
    x0_hat = latents - sigma_t*noise_pred  
    rec_grad = self.get_rec_grad(latents, x0_hat, self.y, self.degradation)  
    noise_pred += sigma_t*rec_grad  
  
# compute the previous noisy sample x_t -> x_{t-1}  
latents = self.scheduler.step(noise_pred, t, latents, **extra_step_kwargs).prev_sample
```

```

def get_rec_grad(self, x, x_hat, y, degradation):

    den_rec= degradation(x_hat)

    if len(y.shape)==3:
        dim=(1,2)
    elif len(y.shape)==2:
        dim=1

    #norm=torch.linalg.norm(y-den_rec,dim=dim, ord=2)

    norm = torch.nn.functional.mse_loss(
        den_rec,
        y,
        reduction="sum"
    )

    rec_grads=torch.autograd.grad(outputs=norm,
                                  inputs=x)
    rec_grads=rec_grads[0]

    audio_vae_length = int(self.transformer.config.sample_size) * self.vae.hop_length

    normguide=torch.linalg.norm(rec_grads)/audio_vae_length**0.5

    #normalize scaling
    #s=self.xi/(normguide*t_i+1e-6)
    s=self.xi/(normguide+1e-6)

    return s*rec_grads

```


Modification du modèle

On veut que le transformer trouve directement $\mathbb{E}[x_0 \mid x(t), y]$

```
# 0. Self-Attention
norm_hidden_states = self.norm1(hidden_states)

attn_output = self.attn1(
    norm_hidden_states,
    attention_mask=attention_mask,
    rotary_emb=rotary_embedding,
)

hidden_states = attn_output + hidden_states

# 2. Cross-Attention
norm_hidden_states = self.norm2(hidden_states)

attn_output = self.attn2(
    norm_hidden_states,
    encoder_hidden_states=encoder_hidden_states,
    attention_mask=encoder_attention_mask,
)

hidden_states = attn_output + hidden_states
```

```
# 2.5 Cross-Attn on Audio
self.norm25 = nn.LayerNorm(dim, norm_eps, True)

self.attn25 = Attention(
    query_dim=dim,
    cross_attention_dim=cross_attention_dim,
    heads=num_attention_heads,
    dim_head=attention_head_dim,
    kv_heads=num_key_value_attention_heads,
    dropout=dropout,
    bias=False,
    upcast_attention=upcast_attention,
    out_bias=False,
    processor=StableAudioAttnProcessor2_0(),
)

# 3. Feed-forward
self.norm3 = nn.LayerNorm(dim, norm_eps, True)
self.ff = FeedForward(
    dim,
    dropout=dropout,
    activation_fn="swiglu",
    final_dropout=False,
    inner_dim=ff_inner_dim,
    bias=True,
)
```

```
self.cross_attention_proj = nn.Sequential(  
    nn.Linear(cross_attention_input_dim, cross_attention_dim, bias=False),  
    nn.SiLU(),  
    nn.Linear(cross_attention_dim, cross_attention_dim, bias=False),  
)  
  
# Un embedding de query peut faire autant de produits scalaires avec key et ainsi ajouter autant de values correspondantes,  
# donc on peut avoir autant d'embeddings qu'on veut pour encoder notre audio inpaint  
# par contre key et query doivent être de la même dimension donc il faut que num_channels_vae passe à cross_attention_dim  
self.cross_attention_proj_audio = nn.Sequential(  
    nn.Linear(cross_attention_input_dim_audio, cross_attention_dim, bias=False),  
    nn.SiLU(),  
    nn.Linear(cross_attention_dim, cross_attention_dim, bias=False),  
)
```

```
noise_pred = self.transformer(  
    latent_model_input,  
    t.unsqueeze(0),  
    encoder_hidden_states=text_audio_duration_embeds,  
    # (B,C,T) -> (B,T,C) la cross_attn attend le nombre de channels en dernier  
    audio_hidden_states = encoded_audio.permute(0, 2, 1),  
    global_hidden_states=audio_duration_embeds,  
    rotary_embedding=rotary_embedding,  
    return_dict=False,  
)[0]
```

Annexe

Inpainting

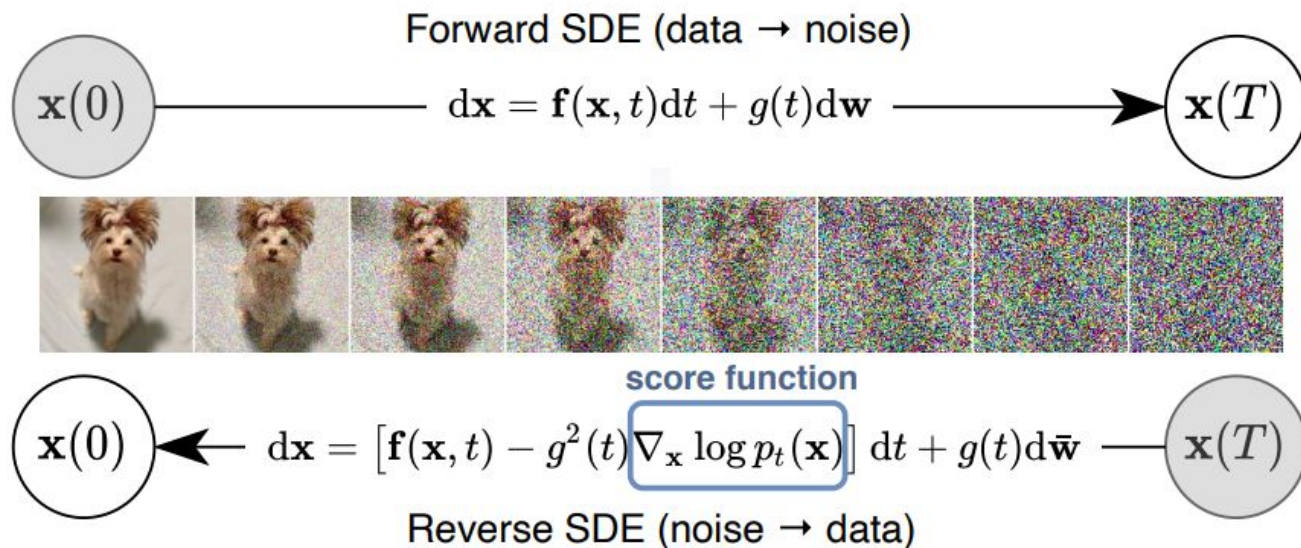
Waveform : [0.2,0.3, 0,0 ,0.7,-0.8]

Masque : [1,1,0,0,1,1]

Encodage de la waveform dans le vae : [0.25,0,-0.1]

Masque réduit à la taille du vae : [1,0,1]

SDE



Les différentes méthodes de diffusion (DDPM, SMLD) correspondent à une certaine SDE forward, dont on discrétise le reverse process

Calculer le score

On peut calculer le score conditionnel

$$\mathbf{x}_\tau = \mathbf{x}_0 + \sigma(\tau)\boldsymbol{\epsilon}, \text{ where } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad p_\tau(x|x_0) \sim \mathcal{N}(x_0, \sigma_\tau I_d)$$

On peut donc déterminer le score conditionnel

$$\nabla_x p_\tau(x|x_0) = \frac{-(x-x_0)}{\sigma^2}$$

Pour avoir le score marginal il faut faire la moyenne sur les x_0

$$\nabla_x \log p_\tau(x) = \mathbb{E}_{x_0|x} [\nabla_x \log p(x | x_0)] = \mathbb{E}_{x_0|x} \left[-\frac{x - x_0}{\sigma^2} \right] = \frac{\mathbb{E}[x_0 | x] - x}{\sigma^2}$$

Il faut donc trouver le x_0 moyen qui en étant bruité à l'étape t donne x

$$\mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} [\lambda(\tau) \|D_\theta(\mathbf{x}_0 + \sigma(\tau)\boldsymbol{\epsilon}, \tau) - \mathbf{x}_0\|_2^2]$$

Probability flow ODE

Pour toute forward SDE, on peut trouver une ODE qui a les mêmes densités marginales.

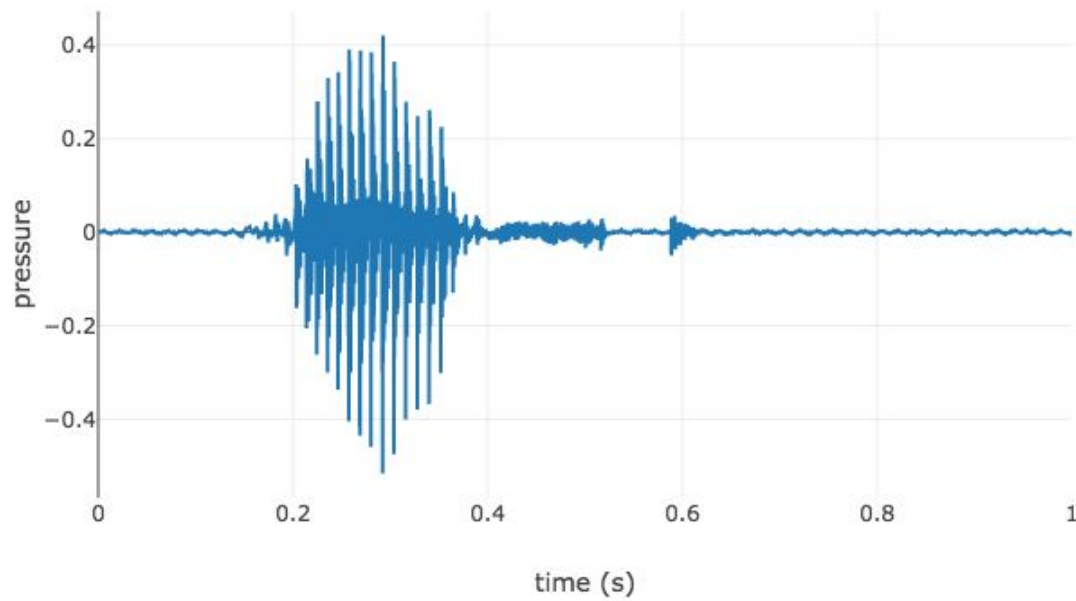
$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt \qquad d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

En intégrant, on a toujours à t la même $p_t(\mathbf{x})$

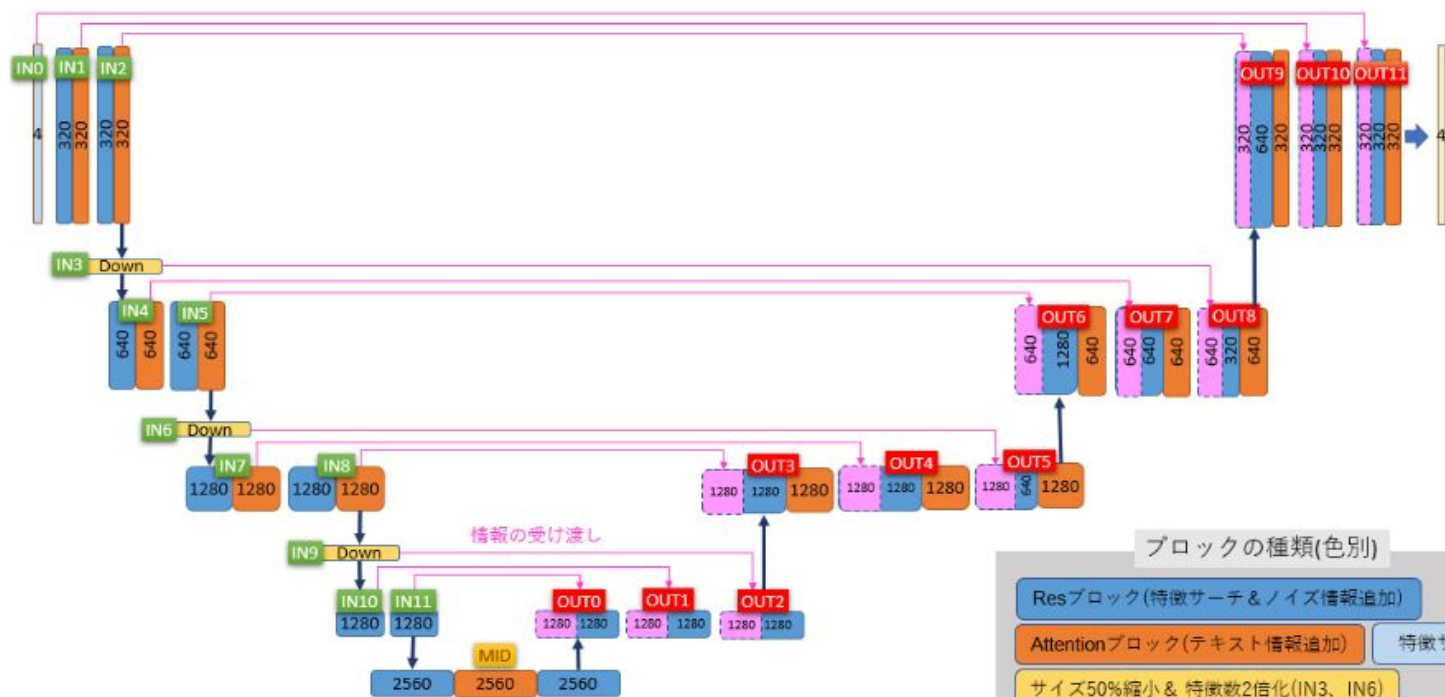
Ainsi au lieu d'intégrer une reverse SDE de $t = 0$ à T , on peut intégrer l'ODE de $t=T$ à 0 .

Les waveforms

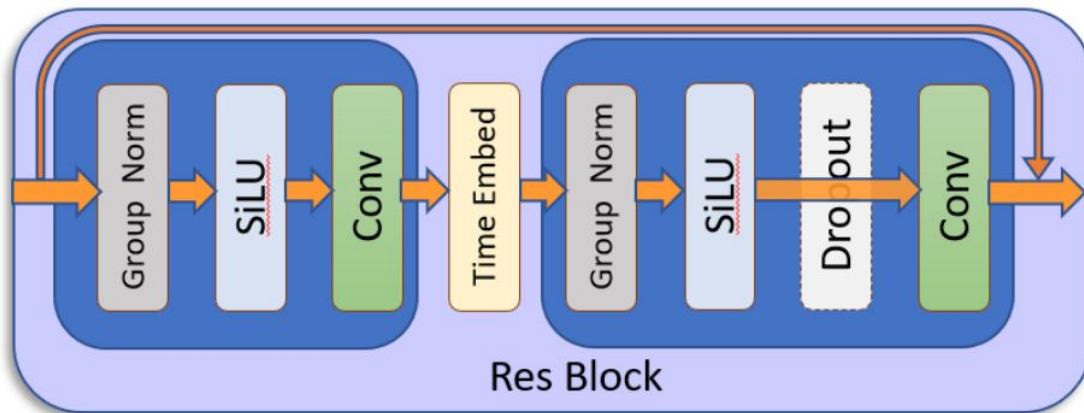
Time domain



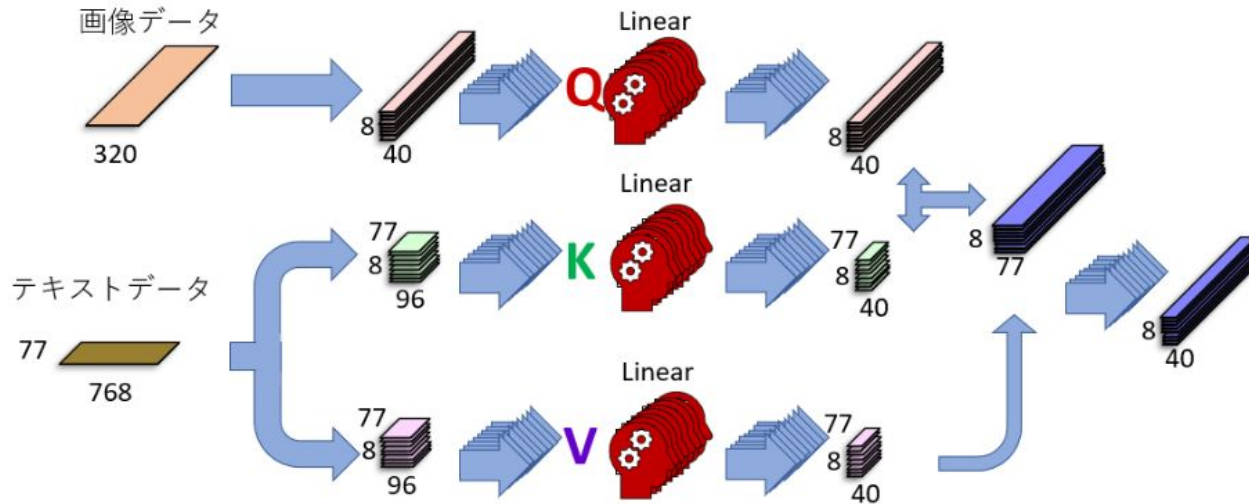
UNet



ResBlock



AttentionBlock



L'attention croisée provient du texte contenant les lettres K et V.

